

FIT3182 Big Data Management and Processing

Semester 1, 2026

Assignment 2 Specification

MongoDB & Streaming

Unit	FIT3182: Big Data Management and Processing	Assignment marks	Assignment - 2: 20 marks Assignment - 2b: 10 marks
Assessment Components	Code Implementation 20 marks	Code Demo & Interview	Code Demonstration & Interview 10 marks
Release Date	21/04/2026 (Tuesday)	Due Date	25/05/2026 11:55 PM (Monday)
Mode	Code implementation & demonstration - Pair (each member submits the assignment on Moodle) Code interview - Individual	Submission Format	Source code, README, outputs

Teaching Team:

- A/Prof. David Taniar (Chief Examiner) | david.taniar@monash.edu
- Dr Yin Yin Low (Unit Coordinator, Lecturer) | low.yinyin@monash.edu
- Dr Vishnu Monn (Unit Coordinator, Lecturer) | vishnu.monnn@monash.edu
- Henry Quan Bi Pay | quan.pay@monash.edu
- Yap Jun Kang | jun.yap@monash.edu
- Chai Wai Jin | chai.waijin@monash.edu
- Andrew Ooi | yooi0010@student.monash.edu

School of Information Technology, Monash University, Malaysia

1. Overview

This assignment assesses practical knowledge and implementation capability in MongoDB data modelling, event streaming, stateful processing, and analytical visualisation for a realistic traffic-monitoring scenario.

Students are required to design, implement, execute, and explain an end-to-end stream processing application using Apache Kafka, Apache Spark Structured Streaming, and MongoDB. The assignment focuses on volume, variety, and velocity in Big Data, with emphasis on data modelling, streaming correctness, state management, sink integration, code quality, and technical justification.

Students work in pairs for this assignment. Students can form their own pairs, but they should be from the same applied group. This is to streamline code demonstration in Week 12 during the applied session. Each pair can share the same code, output and documentation. However, each student should submit the assignment on Moodle. Additionally, the interview session is individual.

2. Learning Outcomes

- Design appropriate MongoDB collections, indexes, and data retention policies for a stream analytics workload.
- Implement Kafka event producers and Spark Structured Streaming logic for joining multiple input streams.
- Apply state management and rule-based detection logic to identify violations from temporal event data.
- Integrate streaming outputs into MongoDB with clear consideration for correctness, efficiency, and idempotency.
- Develop clear technical visualisations and justify the operational value of analytical outputs.
- Communicate implementation decisions and defend design trade-offs during a technical demonstration and interview.

3. Scenario Background

The assignment is framed around an Automated Awareness Safety System (AWAS)-style point-to-point traffic monitoring environment. Multiple roadside cameras record vehicle passages at fixed checkpoints. Camera events are streamed into a real-time processing application that must determine whether a vehicle has violated either the instantaneous speed limit at a camera or the average speed limit across a monitored road segment.

Students must design the storage model, simulate the incoming event streams, implement the online processing logic, and produce clear analytical outputs that support operational monitoring and enforcement use cases.

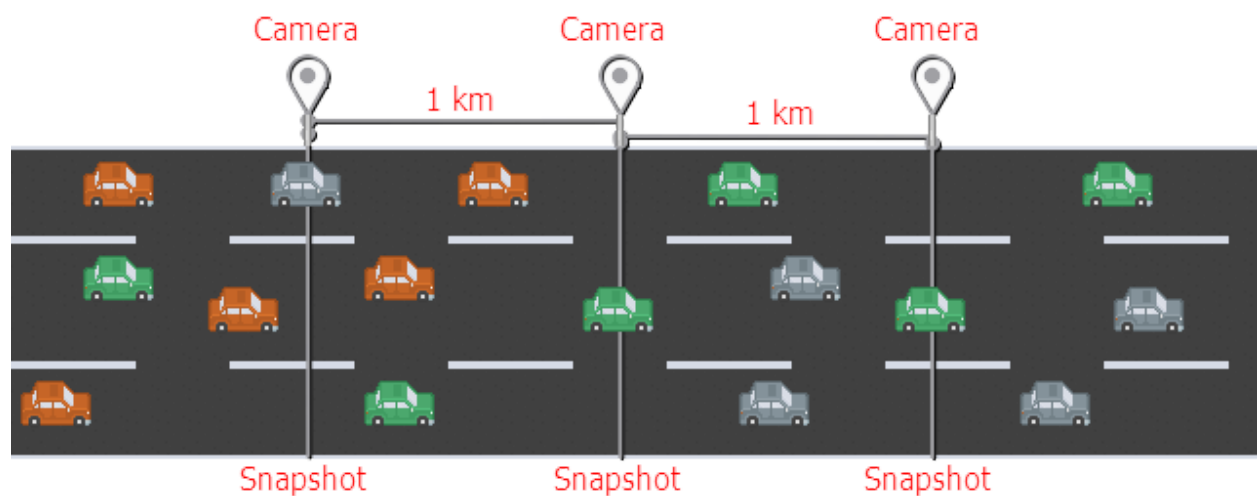


Figure 1. AWAS overview — Conceptual overview of data storage, streaming application, and visualisation components.

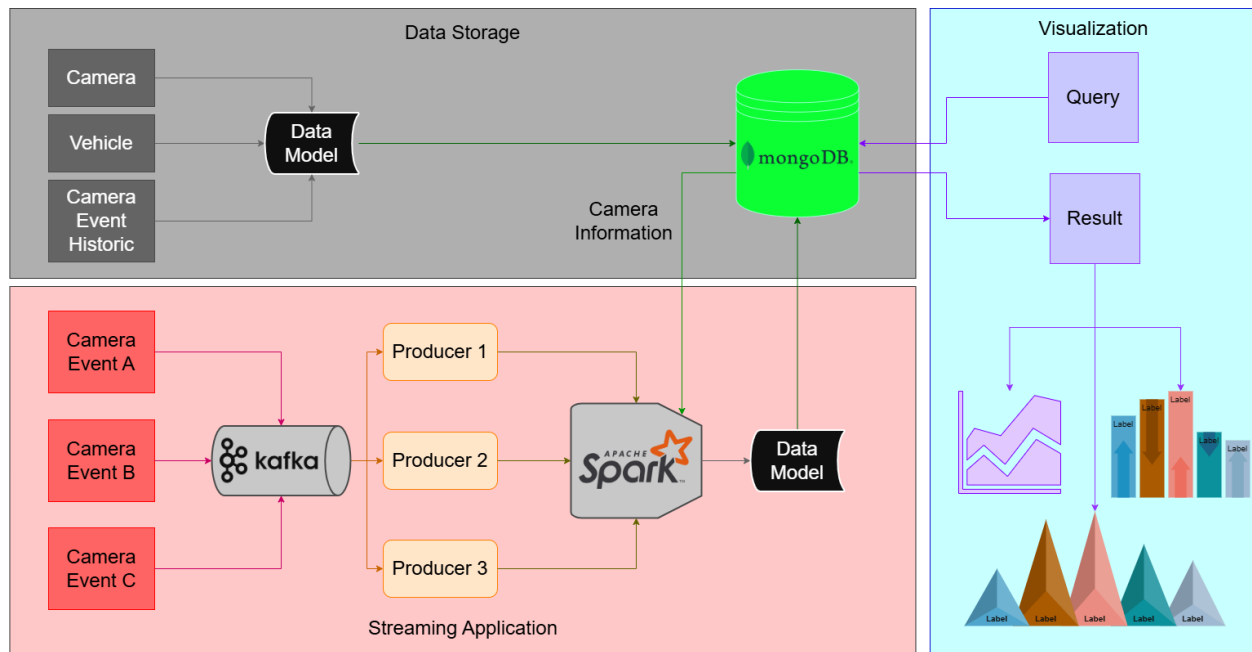


Figure 2. Camera segment example — Illustrative multi-camera checkpoint layout used to derive point and average-speed violations.

4. Dataset Context

This assignment uses an AWAS-inspired traffic monitoring scenario. The following dataset files are provided and should be used as the primary input source unless a task explicitly states otherwise.

File	Purpose
vehicle.csv	Vehicle master data containing ownership and registration details.
camera.csv	Static camera metadata, including position and speed-limit definitions.
camera_event_A.csv	Camera event stream for Producer A.
camera_event_B.csv	Camera event stream for Producer B.
camera_event_C.csv	Camera event stream for Producer C.

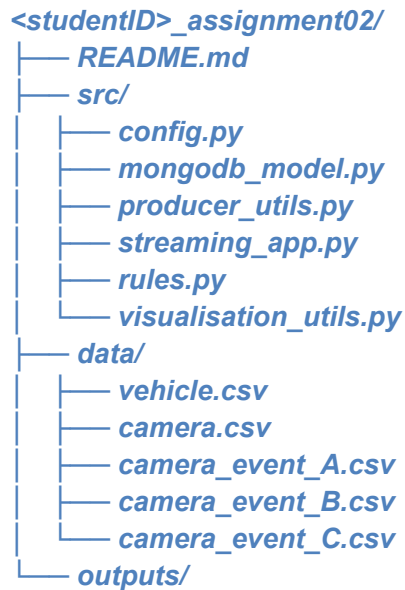
Core fields expected in the event streams include event_id, batch_id, car_plate, camera_id, timestamp, and speed. Students may enrich the streamed payload with producer-specific metadata where appropriate to support traceability and debugging.

5. Technical and Execution Requirements

- All tasks must be implemented in Python using Jupyter notebooks as the primary submission interface. Supporting .py modules may be included where appropriate.
- The submitted solution must be executable and sufficiently organised for the teaching team to reproduce the workflow.
- Students must not hardcode final answers or submit static outputs as a substitute for actual processing logic.
- Streaming solutions should clearly demonstrate event ingestion, join logic, state handling, violation detection, and MongoDB sink integration.

- The submitted notebooks should include sufficient markdown, comments, and evidence to explain the implementation and results.
- When using external libraries, students must document installation requirements and ensure reproducibility.

Recommended submission structure



6. Assignment Task Requirements

Assignment 2

Task 1: MongoDB Data Model (2 marks)

- Task 1.1 Collection Design (1 mark): Design at least the Vehicle, Camera, and Violation collections. For each collection, provide a concise description, document schema, sample document, indexes, shard-key strategy (if any), and retention policy (if applicable).
- Task 1.2 Collection Relationship (1 mark): Explain the relationships between collections and justify whether you embed or reference data. Your discussion should consider read/write patterns, duplication versus join cost, and consistency requirements.

Task 2: Streaming Application (16 marks)

- Task 2.1.1 Producer Implementation (2 marks): Implement Kafka producers for camera_event_A.csv, camera_event_B.csv, and camera_event_C.csv. Producers should publish batches every n seconds based on batch_id and include suitable metadata for source identification.
- Task 2.1.2 Streaming Join Logic and State Management (6 marks): Implement a Spark Structured Streaming application that consumes producer outputs, joins relevant events across streams, manages buffering/state, handles unmatched or expired records appropriately, and supports documentation of cleanup policies.
- Task 2.1.3 Sink Integration with MongoDB (2 marks): Persist detected violations to MongoDB with attention to correctness, retry handling, and efficient writes.

- Task 2.1.4 Violation Detection (2 marks): Detect violations when instantaneous speed exceeds the relevant camera limit, or when the average speed across a segment exceeds the ending camera limit. Where multiple violations occur for the same car on the same day, records should be merged appropriately into a single daily violation record.
- Task 2.2.1 Data Visualisation – Correctness, Plot Design and Clarity (2 marks): Produce line charts or equivalent visual outputs that show violation counts and speed patterns over arrival time. Plots should be labelled and technically interpretable.
- Task 2.2.2 Interesting Point Annotation (2 marks): Mark notable points such as maxima, minima, peaks, or sudden changes. Additional analytical insight beyond basic extrema is encouraged. Explain why the chosen visualisations and annotations matter operationally.

Task 3: Documentation and Code Quality (2 marks)

- Task 3.1 Code Comments (1 mark): Ensure comments/docstrings are where appropriate.
- Task 3.2 Narrative Explanation (1 mark): Add markdown cells, workflow diagrams, tables, and concise explanations that communicate the end-to-end algorithm, important parameters, and implementation rationale.

Assignment 2b

Task 4: Code Demonstration and Interview (10 marks)

- Students must attend the allocated presentation, code demonstration, and interview session. The demonstration should show that the submitted code can be executed and that it can be explained. The interview (individual) assesses understanding of Kafka, Spark, MongoDB, the implemented logic, and the submitted code base.

7. Violation Detection Rules

- A vehicle is flagged when its instantaneous speed exceeds the speed limit of the recording camera.
- A vehicle is also flagged when its average speed across a monitored segment exceeds the speed limit of the ending camera.
- Only relevant event pairs or sequences should be matched; incorrect camera order or invalid timing relationships must not produce false positives.
- Students should explicitly document any assumptions made for buffering, watermarking, time tolerance, or offset handling.

8. Submission Requirements

- Students must submit the main notebook, any producer notebooks or scripts, any supporting Python modules, a README, and output evidence.
- Submission should be packaged in a single compressed folder clearly named using the required student identifier convention.
- All outputs included in the submission must be generated by the submitted code.
- Any use of generative AI must be declared clearly in the submitted work, including the tool, purpose, and prompt or prompt summary.

9. Academic Integrity and Late Submission

- Each pair must ensure that the submitted work reflects their own accountable academic effort and that they can explain it during the interview.
- Late penalties, extensions, and special consideration are to be handled in accordance with official University procedures and Moodle guidance.

10. Assessment Task Structure Distribution

Task	Marks
Task 1: MongoDB Data Model	2
Task 2: Streaming Application	16
Task 3: Documentation and Code Quality	2
Task 4: Code Demonstration and Interview	10
Total Assignment Weight	30 marks or 30% of the total unit marks